ASSIGNMENT





NAME: AZHAR MEHMOOD
REG.NO: FA23-BAI-039

Technical Report

Custom Reinforcement Learning Agent Implementation using Farama Gymnasium

CliffWalking-v1 | Q-Learning (from scratch)

GitHub Repository: https://github.com/azhar-fullstack/RL-Assignment-CliffWalking-QLearning

1. Problem Description

For this assignment I picked CliffWalking-v1 from Gymnasium. It is a small 4x12 grid (48 states), so I could implement the algorithm myself without neural networks. The agent starts at the bottom-left and must reach the goal at the bottom-right. A cliff sits along the bottom row; stepping into it gives a big penalty and sends the agent back to the start. Actions are: 0 = Up, 1 = Right, 2 = Down, 3 = Left.

Every normal step gives reward -1. Falling into the cliff gives -100. Reaching the goal ends the episode. The best safe path is 13 steps, so the best score is -13. A longer upper path is safer in some sense but gets a worse (more negative) score. That tradeoff is why this environment is useful for testing RL.

2. Methodology

2.1 Algorithm

I implemented Q-Learning with an epsilon-greedy policy. I keep a table Q(s, a) for every state-action pair. Most of the time the agent picks the best action from the table, but with probability epsilon it picks a random action to explore. After each step:

$$Q(s,a) = Q(s,a) + \text{alpha} * [\, r + \text{gamma} * \max Q(s', a') - Q(s,a) \,]$$

Here alpha is the learning rate and gamma is the discount factor. Epsilon starts at 1.0 and decays each episode down to 0.01. Using max over next actions makes this the normal off-policy Q-Learning update.

2.2 Code Structure

I split the project into three files. agent.py has the QLearningAgent class (action selection, Q update, epsilon decay, save/load). utils.py has training, evaluation, plots, and GIF rendering. main.py runs the full pipeline. The environment is created with gymnasium.make("CliffWalking-v1"). After training I save models/q_table.npy and results/agent_demo.gif.

3. Experimental Setup and Hyperparameters

I trained in Python with Gymnasium. The table below shows the final settings used for the saved model and GIF. For sensitivity tests I ran 3000 episodes again and changed one setting at a time with the same seed.

Hyperparameter	Symbol	Value	Why I chose it
Learning rate	alpha	0.5	0.1 was slow; 0.9 less

			stable
Discount factor	gamma	0.99	Care more about reaching goal
Start epsilon	eps0	1.0	Start fully random
Min epsilon	eps_min	0.01	Keep a little exploration
Epsilon decay	eps_decay	0.995	Decay once per episode
Training episodes	N	5000	Main training run
Max steps / episode	T	200	Stop very long episodes
Eval episodes	-	100	Greedy policy test
Seed	-	42	Fair comparison

4. Results and Discussion

After 5000 episodes, greedy evaluation over 100 episodes gave average return -13.0 (no variance) and 100% success. So the agent learned the optimal 13-step path. In Figure 1, reward improves mainly in the first 1000-2000 episodes, then becomes more stable as epsilon gets small. Figure 2 shows higher values along the good path and lower values near the cliff.



Figure 1. Reward, success rate, and epsilon during training.

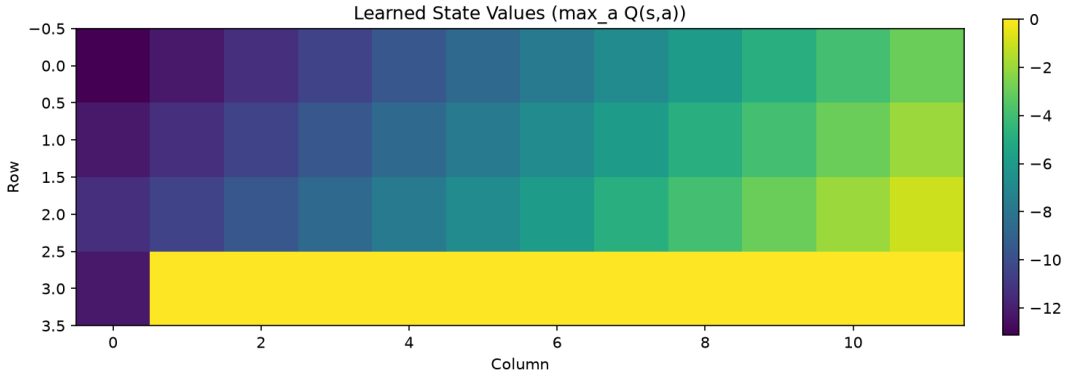


Figure 2. Max Q-value for each cell on the 4x12 map.

In the hyperparameter check (Figure 3), alpha = 0.5 beat alpha = 0.1 under the same episode budget. Gamma = 0.99 worked fine. Slow epsilon decay (0.999) kept exploration high for longer, so late training return stayed worse. Learning rate and epsilon decay mattered the most in my runs.

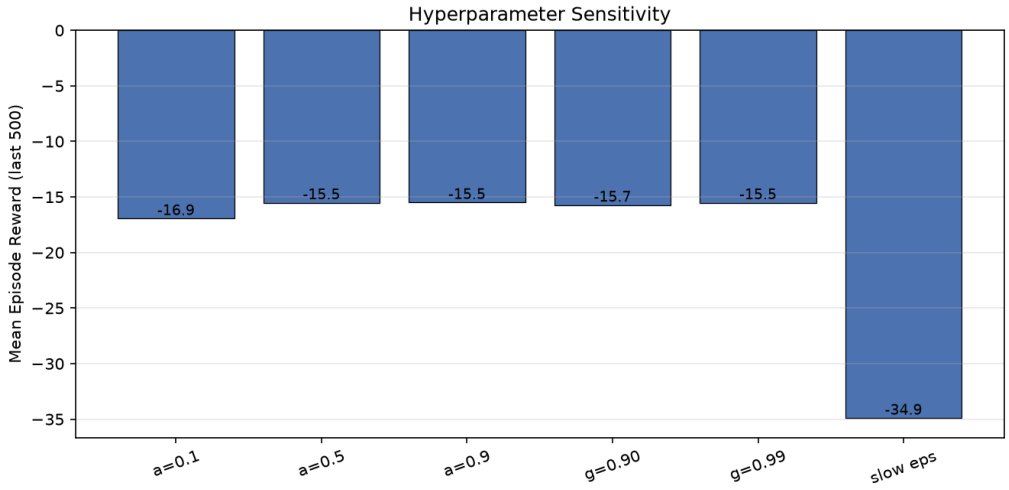


Figure 3. Average return in the last 500 episodes for different settings.

4.1 Problems I Noticed

Early on the agent falls a lot because epsilon is high; that looks bad in the reward curve but is normal while exploring. It can also learn a longer upper-row path that is safer but worse than -13. This table method works here, but not for continuous tasks like CartPole without something like DQN. The demo GIF shows the final greedy policy reaching the goal without falling.

5. Conclusion

I built a from-scratch Q-Learning agent on CliffWalking-v1, trained it, plotted the curves, saved the Q-table, and recorded a short GIF. In evaluation it reached the optimal score (-13). Changing alpha, gamma, and epsilon decay clearly changed learning speed and final return. Next I would try SARSA or Double Q-Learning. Main files are main.py, agent.py, utils.py, notebooks/training.ipynb, models/q_table.npy, and results/agent_demo.gif.